

Chapter 3 — Two Pointers

0. What You Will Learn in This Chapter

Two Pointers is one of the most important coding interview patterns for arrays and strings.

It is used when you maintain **two positions** in the input and move them intelligently to avoid unnecessary work.

By the end of this chapter, you should be able to solve problems involving:

- Sorted pair search
- Palindrome checking
- Reversing arrays/strings
- Removing elements in-place
- Removing duplicates from sorted arrays
- Moving zeroes
- Finding best pair from two ends
- 3Sum / 4Sum-style problems
- Sorted squares
- Dutch National Flag partitioning
- Trapping Rain Water
- Backspace string comparison
- Boats to Save People

Core LeetCode problems covered in this chapter:

- 1.1. Valid Palindrome
 - 2.2. Two Sum II — Input Array Is Sorted
 - 3.3. Reverse String
 - 4.4. Remove Duplicates from Sorted Array
 - 5.5. Remove Element
 - 6.6. Move Zeroes
 - 7.7. Container With Most Water
 - 8.8. 3Sum
 - 9.9. Squares of a Sorted Array
 - 10.10. Sort Colors
 - 11.11. Trapping Rain Water
 - 12.12. Valid Palindrome II
 - 13.13. Boats to Save People
 - 14.14. Backspace String Compare
-

1. What Is the Two Pointers Pattern?

The Two Pointers pattern uses two variables, usually indices, to scan through an array or string.

The classic version starts one pointer at the beginning and one at the end:

```
int left = 0;
int right = nums.size() - 1;

while (left < right) {
```

```
// process nums[left] and nums[right]

left++;
right--;
}
```

But two pointers can appear in several forms.

1.1 Opposite-Direction Pointers

Pointers start at both ends and move inward.

```
int left = 0;
int right = nums.size() - 1;

while (left < right) {
    // compare or process both ends
}
```

Used for:

- Palindromes
- Reversing
- Two Sum in sorted array
- Container With Most Water
- Trapping Rain Water
- Sorted Squares

1.2 Same-Direction Pointers

Both pointers move left to right.

Common version:

```
int write = 0;

for (int read = 0; read < nums.size(); read++) {
    if (/* keep nums[read] */) {
        nums[write] = nums[read];
        write++;
    }
}
```

Used for:

- Remove Element
- Move Zeroes
- Remove Duplicates
- Compacting arrays

1.3 Fixed Index + Two Pointers

One index is fixed, and two pointers search the remaining part.

Used in:

- 3Sum
- 4Sum
- 3Sum Closest

Example structure:

```
sort(nums.begin(), nums.end());

for (int i = 0; i < nums.size(); i++) {
    int left = i + 1;
    int right = nums.size() - 1;

    while (left < right) {
        // use nums[i], nums[left], nums[right]
    }
}
```

2. Why Two Pointers Exists

Many pair problems are solved by checking all pairs.

Example:

```
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        // check nums[i], nums[j]
    }
}
```

This is $O(n^2)$.

Two pointers can reduce many of these to $O(n)$, especially when the input is sorted.

Example:

```
nums = {1, 2, 4, 6, 8}
target = 10
```

Start:

```
left = 0 // nums[left] = 1
right = 4 // nums[right] = 8
```

Sum is 9.

Since the array is sorted and sum is too small, move left right to increase the sum.

Now:

```
nums[left] = 2
nums[right] = 8
sum = 10
```

Found.

Sorting or sorted order gives direction.

3. Core Need / Why Use Two Pointers Over Other Patterns?

Two pointers is useful when moving one pointer lets you safely eliminate many possibilities.

3.1 Why Not Brute Force?

Brute force checks every possible pair.

Two pointers often skips many impossible pairs.

For sorted pair sum:

- If sum is too small, every pair with current left and smaller/equal right is too small.
- So move left.
- If sum is too large, every pair with current right and larger/equal left is too large.
- So move right.

This eliminates many pairs at once.

3.2 Why Not Hashing?

Hashing is great for unsorted Two Sum.

But if the array is already sorted, two pointers is often better because:

- $O(n)$ time
- $O(1)$ extra space
- no hash table overhead
- uses sorted order directly

Example:

LeetCode 167 Two Sum II should be solved with two pointers.

3.3 Why Not Sorting?

Sometimes sorting enables two pointers.

But if original order matters, sorting is wrong.

Example:

Container With Most Water:

height = {1,8,6,2,5,4,8,3,7}

Sorting destroys index distances, so it is invalid.

3.4 Why Not Sliding Window?

Sliding window is technically a two-pointer technique, but it maintains a whole contiguous range.

Two pointers usually focuses on:

- pair of elements
- symmetry
- in-place compaction
- sorted search
- boundary movement

Sliding window usually focuses on:

- longest/shortest subarray
 - substring
 - frequency/state inside a window
-

4. When to Use Two Pointers

Use Two Pointers when you see these clues.

4.1 Sorted Array + Pair Condition

Problem says:

- sorted array
- find two numbers
- pair sum
- target
- closest sum
- return pair

Examples:

- Two Sum II
 - 3Sum
 - 4Sum
 - 3Sum Closest
-

4.2 Palindrome / Symmetry

Problem says:

- palindrome
- compare from both ends
- ignore punctuation
- almost palindrome
- reverse

Examples:

- Valid Palindrome
 - Valid Palindrome II
 - Reverse String
 - Reverse Vowels of a String
-

4.3 In-Place Modification

Problem says:

- remove element
- remove duplicates
- move zeroes
- modify array in-place
- return new length

Examples:

- Remove Element
- Remove Duplicates from Sorted Array
- Move Zeroes
- String Compression

4.4 Best Pair From Ends

Problem asks:

- maximum area
- minimum boats
- pair lightest with heaviest
- largest/smallest pair

Examples:

- Container With Most Water
 - Boats to Save People
-

4.5 Sorted Extremes

Problem involves sorted array but negative values affect order.

Example:

`nums = {-4, -1, 0, 3, 10}`

Squares at both ends may be largest.

Use two pointers.

Problem:

- Squares of a Sorted Array
-

4.6 Partitioning

Problem asks:

- separate 0s, 1s, 2s
- move even before odd
- partition around values

Examples:

- Sort Colors
 - Sort Array By Parity
 - Move Zeroes
-

5. When NOT to Use Two Pointers

5.1 Array Is Unsorted and Pointer Movement Has No Meaning

For pair sum in unsorted array:

`nums = {5, 1, 8, 2}`
`target = 10`

Moving left/right does not predictably increase or decrease sum.

Use hashing or sort first if allowed.

5.2 Original Indices Matter and Sorting Would Lose Them

LeetCode 1 Two Sum requires original indices.

Hash map is usually better.

If you sort, you must store original indices.

5.3 Problem Is About Subsequence

Subsequence is not contiguous and not necessarily from both ends.

Use:

- DP
- greedy
- binary search
- backtracking

depending on problem.

5.4 Need Fast Arbitrary Lookup

If you need to know if any arbitrary value exists, hashing is better.

5.5 Need All Combinations

If the problem asks to generate all subsets/permutations, use backtracking.

6. Two Pointers vs Sliding Window

Main idea	Move two positions intelligently	Maintain a contiguous window
Common use	Pairs, symmetry, in-place changes	Subarrays/substrings
Direction	Often opposite ends	Usually both move forward
State	Pair comparison or placement	Window sum/frequency/count
Example	Two Sum II	Longest Substring Without Repeating Characters
Typical movement	left++ or right--	expand right, shrink left

Sliding window is a specialized form of two pointers, but it is important enough to treat separately.

7. Mental Models

7.1 Squeeze From Both Ends

For opposite-end pointers:

left -> <- right

Each movement removes impossible choices.

Used for:

- palindrome
- Two Sum sorted

- Container With Most Water
-

7.2 Smallest + Largest Pair

In sorted arrays:

- left is smallest remaining
- right is largest remaining

If sum is too small, increase the small side.

If sum is too large, decrease the large side.

7.3 Read/Write Conveyor Belt

For in-place compaction:

- read scans every item on conveyor belt
- write places valid items at the front

Example:

read: scans all elements

write: next valid position

7.4 Symmetry Mirror

For palindrome:

`s[left]` must match `s[right]`

If they match, move inward.

If they do not, invalid or try one deletion.

7.5 Boundary Greedy

For Container With Most Water:

The area is limited by the shorter side.

Move the shorter side because moving the taller side cannot improve the limiting height.

8. Pattern Recognition Checklist

Ask:

2. Is the array sorted?
 3. Do I need a pair or triplet?
 4. Can moving one pointer safely eliminate possibilities?
 5. Is this a palindrome or mirror problem?
 6. Do I need to reverse something in-place?
 7. Do I need to remove/compact values in-place?
 8. Can I sort first without losing required order/indices?
 9. Does the problem involve smallest and largest remaining values?
 10. Is the answer based on two boundaries?
 11. Is a sliding window more appropriate because the problem asks about subarrays?
-

9. C++ Tools Needed

9.1 Vector Indexing

```
nums[left]
nums[right]
```

9.2 String Indexing

```
s[left]
s[right]
```

9.3 swap

```
swap(nums[left], nums[right]);
```

Used for:

- reversing
- partitioning
- Sort Colors
- in-place operations

9.4 sort

```
sort(nums.begin(), nums.end());
```

Used before two pointers when original order does not matter.

9.5 Character Helpers

For palindrome problems:

```
isalnum(c)
tolower(c)
```

These come from:

```
#include <cctype>
```

LeetCode usually includes it.

9.6 Important Type Casting

For reverse loops or `right = nums.size() - 1`, if array may be empty, be careful because `nums.size()` is unsigned.

Safer:

```
int right = (int)nums.size() - 1;
```

10. Core Templates

Template 1 — Opposite-Direction Pointers

```
int left = 0;
int right = nums.size() - 1;

while (left < right) {
    // process nums[left] and nums[right]

    left++;
    right--;
}
```

Used for:

- reverse
 - palindrome
 - inward comparison
-

Template 2 — Sorted Two Sum

```
int left = 0;
int right = nums.size() - 1;

while (left < right) {
    int sum = nums[left] + nums[right];

    if (sum == target) {
        return {left, right};
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
```

Template 3 — Palindrome Check

```
bool isPalindrome(string s) {
    int left = 0;
    int right = s.size() - 1;

    while (left < right) {
        if (s[left] != s[right]) {
            return false;
        }
    }
}
```

```
        left++;
        right--;
    }

    return true;
}
```

Template 4 — Reverse Array/String

```
int left = 0;
int right = nums.size() - 1;

while (left < right) {
    swap(nums[left], nums[right]);
    left++;
    right--;
}
```

Template 5 — Read/Write Pointer

```
int write = 0;

for (int read = 0; read < nums.size(); read++) {
    if (/* keep nums[read] */) {
        nums[write] = nums[read];
        write++;
    }
}

return write;
```

Template 6 — Remove Duplicates from Sorted Array

```
if (nums.empty()) {
    return 0;
}

int write = 1;

for (int read = 1; read < nums.size(); read++) {
    if (nums[read] != nums[write - 1]) {
        nums[write] = nums[read];
        write++;
    }
}
```

```
    }  
}  
  
return write;
```

Template 7 — Fixed Index + Two Pointers

```
sort(nums.begin(), nums.end());  
  
for (int i = 0; i < nums.size(); i++) {  
    if (i > 0 && nums[i] == nums[i - 1]) {  
        continue;  
    }  
  
    int left = i + 1;  
    int right = nums.size() - 1;  
  
    while (left < right) {  
        int sum = nums[i] + nums[left] + nums[right];  
  
        if (sum == target) {  
            // store answer  
  
            left++;  
            right--;  
  
            while (left < right && nums[left] == nums[left - 1]) {  
                left++;  
            }  
  
            while (left < right && nums[right] == nums[right + 1]) {  
                right--;  
            }  
        } else if (sum < target) {  
            left++;  
        } else {  
            right--;  
        }  
    }  
}
```

Used for:

- 3Sum
- 4Sum

- 3Sum Closest
-

Template 8 — Fill Result From End

Used when the largest answer comes from either end.

```
vector<int> result(nums.size());

int left = 0;
int right = nums.size() - 1;
int pos = nums.size() - 1;

while (left <= right) {
    if (abs(nums[left]) > abs(nums[right])) {
        result[pos] = nums[left] * nums[left];
        left++;
    } else {
        result[pos] = nums[right] * nums[right];
        right--;
    }

    pos--;
}
```

Used for:

- Squares of a Sorted Array
-

Template 9 — Three-Way Partition

```
int low = 0;
int mid = 0;
int high = nums.size() - 1;

while (mid <= high) {
    if (nums[mid] == 0) {
        swap(nums[low], nums[mid]);
        low++;
        mid++;
    } else if (nums[mid] == 1) {
        mid++;
    } else {
        swap(nums[mid], nums[high]);
        high--;
    }
}
```

Used for:

- Sort Colors

Template 10 — Two Pointers With Left/Right State

```
int left = 0;
int right = height.size() - 1;

int leftMax = 0;
int rightMax = 0;
int answer = 0;

while (left < right) {
    if (height[left] < height[right]) {
        leftMax = max(leftMax, height[left]);
        answer += leftMax - height[left];
        left++;
    } else {
        rightMax = max(rightMax, height[right]);
        answer += rightMax - height[right];
        right--;
    }
}
```

Used for:

- Trapping Rain Water

11. Major Variations of Two Pointers

Pair search sorted	left = 0, right = n - 1	Two Sum II
Palindrome	compare ends	Valid Palindrome
Reverse in-place	swap ends	Reverse String
Read/write compaction	read, write	Remove Element
Best boundary pair	move limiting side	Container With Most Water
Fixed index + two pointers	sort + loop	3Sum

Variation	Setup	Example
Fill result from end	compare extremes	Sorted Squares
Three-way partition	low, mid, high	Sort Colors
Two-sided state	leftMax/rightMax	Trapping Rain Water

12. Full Problem Walkthrough 1 — LeetCode 125: Valid Palindrome

12.1 Problem Summary

Given a string *s*, return true if it is a palindrome after:

- converting uppercase to lowercase
- removing non-alphanumeric characters

Example:

`s = "A man, a plan, a canal: Panama"`

Output:

`true`

Cleaned string:

`amanaplanacanalpanama`

12.2 How to Identify This Pattern

Problem says:

- palindrome
- compare characters from both ends
- ignore certain characters

This is a two-pointer symmetry problem.

12.3 Brute Force Idea

Build a cleaned string, reverse it, compare.

Time: $O(n)$

Space: $O(n)$

This works, but two pointers avoids extra space.

12.4 Optimized Intuition

Use:

`left = 0`

`right = s.size() - 1`

While `left < right`:

12. Skip invalid left characters.
13. Skip invalid right characters.

14. Compare lowercase characters.
 15. Move inward.
-

12.5 C++ Code

```
class Solution {
public:
    bool isPalindrome(string s) {
        int left = 0;
        int right = (int)s.size() - 1;

        while (left < right) {
            while (left < right && !isalnum(s[left])) {
                left++;
            }

            while (left < right && !isalnum(s[right])) {
                right--;
            }

            if (tolower(s[left]) != tolower(s[right])) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }
};
```

12.6 Code Explanation

```
while (left < right && !isalnum(s[left]))
```

Skips punctuation, spaces, symbols.

```
tolower(s[left])
```

Converts uppercase to lowercase for comparison.

```
left++;
right--;
```

Move inward after successful match.

12.7 Dry Run

Input:

s = "A man, a plan, a canal: Panama"

Important comparisons:

A	a	a vs a	yes
m	m	m vs m	yes
a	a	a vs a	yes
n	n	n vs n	yes

Eventually all valid pairs match.

Return true.

12.8 Correctness Explanation

A palindrome must match symmetrically from the outside inward. Non-alphanumeric characters are ignored, so skipping them preserves the valid cleaned sequence. If any matching pair differs, no cleaned palindrome is possible.

12.9 Complexity

Time: $O(n)$

Each pointer moves at most n times.

Space: $O(1)$

12.10 Edge Cases

- Empty string
- Only punctuation
- Mixed uppercase/lowercase
- Digits
- Single character
- Spaces between characters

12.11 Common Mistakes

Mistake 1: Not skipping invalid characters with while

Wrong:

```
if (!isalnum(s[left])) {  
    left++;  
}
```

There may be many invalid characters in a row.

Use while.

Mistake 2: Forgetting case conversion

A and a should match.

12.12 Similar Problems

- Valid Palindrome II
 - Reverse Vowels of a String
 - Reverse Only Letters
 - Palindromic Substrings
-

13. Full Problem Walkthrough 2 — LeetCode 167: Two Sum II — Input Array Is Sorted

13.1 Problem Summary

Given a sorted array numbers, return 1-indexed positions of two numbers that add to target.

Example:

```
numbers = {2, 7, 11, 15}
target = 9
```

Output:

```
{1, 2}
```

13.2 How to Identify This Pattern

Problem says:

- sorted array
- two numbers
- target sum
- return indices

This is classic sorted two pointers.

13.3 Brute Force Idea

Try every pair.

Time: $O(n^2)$

13.4 Optimized Intuition

Use smallest and largest remaining values.

```
sum = numbers[left] + numbers[right]
```

If sum is too small:

```
left++;
```

If sum is too large:

```
right--;
```

13.5 C++ Code

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int left = 0;
        int right = numbers.size() - 1;

        while (left < right) {
            int sum = numbers[left] + numbers[right];

            if (sum == target) {
                return {left + 1, right + 1};
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }

        return {};
    }
};
```

13.6 Code Explanation

```
return {left + 1, right + 1};
```

Problem asks for 1-indexed positions.

```
sum < target
```

Need a larger sum, move left.

```
sum > target
```

Need a smaller sum, move right.

13.7 Dry Run

```
numbers = {2, 7, 11, 15}
```

```
target = 9
```

0	3	2 + 15	17	too large, right--
0	2	2 + 11	13	too large, right--

left	right	values	sum	action
0	1	2 + 7	9	found

Return {1, 2}.

13.8 Correctness Explanation

Because the array is sorted, moving left increases the sum and moving right decreases the sum. Each step removes a set of impossible pairs.

13.9 Complexity

Time: $O(n)$

Space: $O(1)$

13.10 Edge Cases

- Negative numbers
 - Duplicate values
 - Exactly two elements
 - Target at extremes
 - Multiple valid pairs, usually one guaranteed
-

13.11 Common Mistakes

- Returning 0-indexed indices
 - Moving wrong pointer
 - Using this logic on unsorted array
-

13.12 Similar Problems

- Two Sum
 - 3Sum
 - 4Sum
 - 3Sum Closest
-

14. Full Problem Walkthrough 3 — LeetCode 344: Reverse String

14.1 Problem Summary

Reverse a character array in-place.

Example:

<code>s = {'h', 'e', 'l', 'l', 'o'}</code>
--

Output:

<code>{'o', 'l', 'l', 'e', 'h'}</code>
--

14.2 How to Identify This Pattern

Problem says:

- reverse
- in-place
- character array

Use two pointers from both ends.

14.3 C++ Code

```
class Solution {
public:
    void reverseString(vector<char>& s) {
        int left = 0;
        int right = s.size() - 1;

        while (left < right) {
            swap(s[left], s[right]);
            left++;
            right--;
        }
    }
};
```

14.4 Dry Run

s = {'h', 'e', 'l', 'l', 'o'}

0	4	swap h/o	o e l l h
1	3	swap e/l	o l l e h
2	2	stop	o l l e h

14.5 Complexity

Time: O(n)

Space: O(1)

14.6 Edge Cases

- Empty vector
 - One character
 - Even length
 - Odd length
-

14.7 Similar Problems

- Reverse Vowels of a String
- Reverse String II

- Reverse Words in a String III
 - Reverse Only Letters
-

15. Full Problem Walkthrough 4 — LeetCode 26: Remove Duplicates from Sorted Array

15.1 Problem Summary

Given a sorted array, remove duplicates in-place so each unique element appears once.

Return the number of unique elements.

Example:

```
nums = {1, 1, 2}
```

Return:

```
2
```

Front becomes:

```
{1, 2}
```

15.2 How to Identify This Pattern

Problem says:

- sorted array
- remove duplicates
- in-place
- return new length

Use read/write pointer.

15.3 C++ Code

```
class Solution {
public:
    int removeDuplicates(vector<int>& nums) {
        if (nums.empty()) {
            return 0;
        }

        int write = 1;

        for (int read = 1; read < nums.size(); read++) {
            if (nums[read] != nums[write - 1]) {
                nums[write] = nums[read];
                write++;
            }
        }
    }
};
```

```

    }
}

return write;
}
};

```

15.4 Why This Works

The array is sorted, so duplicates are adjacent.

nums[write - 1] is the last unique value kept.

If current value differs from it, current value is new and should be written.

15.5 Dry Run

nums = {0, 0, 1, 1, 1, 2, 2, 3}

1	0	0	skip	1
2	1	0	write 1	2
3	1	1	skip	2
4	1	1	skip	2
5	2	1	write 2	3
6	2	2	skip	3
7	3	2	write 3	4

Return 4.

15.6 Complexity

Time: $O(n)$

Space: $O(1)$

15.7 Edge Cases

- Empty array
- One element
- All duplicates
- No duplicates
- Negative numbers

15.8 Common Mistakes

Mistake 1: Returning write - 1

write is the count of unique elements.

Return write.

[Mistake 2: Forgetting sorted condition](#)

This solution depends on duplicates being adjacent.

15.9 Similar Problems

- Remove Duplicates from Sorted Array II
 - Remove Element
 - Move Zeroes
 - String Compression
-

16. Full Problem Walkthrough 5 — LeetCode 27: Remove Element

16.1 Problem Summary

Remove all occurrences of val in-place and return new length.

Example:

```
nums = {3, 2, 2, 3}
val = 3
```

Return:

```
2
```

Front becomes:

```
{2, 2}
```

16.2 How to Identify This Pattern

Problem says:

- remove certain value
- in-place
- return new length

Use read/write pointer.

16.3 Stable C++ Solution

This preserves order.

```
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int write = 0;

        for (int read = 0; read < nums.size(); read++) {
            if (nums[read] != val) {
                nums[write] = nums[read];
                write++;
            }
        }
    }
};
```

```
    }  
  }  
  
  return write;  
}  
};
```

16.4 Faster Unstable Alternative

If order does not matter:

```
class Solution {  
public:  
    int removeElement(vector<int>& nums, int val) {  
        int i = 0;  
        int n = nums.size();  
  
        while (i < n) {  
            if (nums[i] == val) {  
                nums[i] = nums[n - 1];  
                n--;  
            } else {  
                i++;  
            }  
        }  
  
        return n;  
    }  
};
```

This can reduce writes if val is rare or many removals occur near front.

16.5 Complexity

Stable solution:

- Time: $O(n)$
- Space: $O(1)$

Unstable solution:

- Time: $O(n)$
- Space: $O(1)$

16.6 Common Mistake

Incrementing i after replacing with last element in unstable version.

Wrong because the new value at i has not been checked yet.

17. Full Problem Walkthrough 6 — LeetCode 283: Move Zeroes

17.1 Problem Summary

Move all zeroes to the end while maintaining order of non-zero elements.

Example:

```
nums = {0, 1, 0, 3, 12}
```

Output:

```
{1, 3, 12, 0, 0}
```

17.2 How to Identify This Pattern

Problem says:

- move zeroes
- preserve order
- in-place

Use read/write pointer.

17.3 C++ Code

```
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int write = 0;

        for (int read = 0; read < nums.size(); read++) {
            if (nums[read] != 0) {
                nums[write] = nums[read];
                write++;
            }
        }

        while (write < nums.size()) {
            nums[write] = 0;
            write++;
        }
    }
};
```

17.4 Dry Run

```
nums = {0, 1, 0, 3, 12}
```

0	0	skip	0	{0,1,0,3,12}
1	1	write 1	1	{1,1,0,3,12}
2	0	skip	1	{1,1,0,3,12}
3	3	write 3	2	{1,3,0,3,12}
4	12	write 12	3	{1,3,12,3,12}

Fill rest:

{1, 3, 12, 0, 0}

17.5 Swap Version

```
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int write = 0;

        for (int read = 0; read < nums.size(); read++) {
            if (nums[read] != 0) {
                swap(nums[write], nums[read]);
                write++;
            }
        }
    }
};
```

This also preserves relative order of non-zero elements.

17.6 Complexity

Time: $O(n)$

Space: $O(1)$

17.7 Edge Cases

- All zeroes
- No zeroes
- One element
- Zeroes at front
- Zeroes at end

17.8 Similar Problems

- Remove Element

- Remove Duplicates
 - Sort Array By Parity
 - Apply Operations to an Array
-

18. Full Problem Walkthrough 7 — LeetCode 11: Container With Most Water

18.1 Problem Summary

Given height, choose two lines that form a container with the x-axis.

Return maximum water area.

Example:

```
height = {1, 8, 6, 2, 5, 4, 8, 3, 7}
```

Output:

```
49
```

18.2 How to Identify This Pattern

Problem asks:

- choose two boundaries
- maximize area
- area depends on width and smaller height

This suggests two pointers from both ends.

18.3 Brute Force Idea

Check every pair.

Time: $O(n^2)$

18.4 Optimized Intuition

Area:

```
width * min(height[left], height[right])
```

Start with maximum width.

The shorter line limits area.

If we move the taller line:

- width decreases
- limiting height does not improve

So move the shorter line.

18.5 C++ Code

```
class Solution {  
public:  
    int maxArea(vector<int>& height) {
```

```

int left = 0;
int right = height.size() - 1;

int best = 0;

while (left < right) {
    int width = right - left;
    int h = min(height[left], height[right]);
    int area = width * h;

    best = max(best, area);

    if (height[left] < height[right]) {
        left++;
    } else {
        right--;
    }
}

return best;
}
};

```

18.6 Dry Run

height = {1,8,6,2,5,4,8,3,7}

0	8	1,7	8	8	8	left
1	8	8,7	7	49	49	right
1	7	8,3	6	18	49	right
1	6	8,8	5	40	49	right
1	5	8,4	4	16	49	right

Best remains 49.

18.7 Correctness Explanation

At each step, the shorter line limits the water height. Since moving either pointer reduces width, only moving the shorter line can possibly find a taller limiting boundary. Moving the taller line cannot improve the current limiting height.

18.8 Complexity

Time: $O(n)$
Space: $O(1)$

18.9 Common Mistakes

- Sorting heights, which destroys distances
 - Moving taller pointer
 - Forgetting width matters
 - Thinking local tallest lines always win
-

18.10 Similar Problems

- Trapping Rain Water
 - Boats to Save People
 - Max Area of Island, unrelated but similar wording
 - Most Water style boundary problems
-

19. Full Problem Walkthrough 8 — LeetCode 15: 3Sum

19.1 Problem Summary

Given an integer array, return all unique triplets that sum to zero.

Example:

```
nums = {-1, 0, 1, 2, -1, -4}
```

Output:

```
{{-1, -1, 2}, {-1, 0, 1}}
```

19.2 How to Identify This Pattern

Problem asks:

- triplets
- unique results
- sum equals target

Sort the array, fix one number, then solve Two Sum with two pointers.

19.3 Brute Force Idea

Try all triplets.

Time: $O(n^3)$

19.4 Optimized Intuition

Sort first.

For each i :

```
target for pair = -nums[i]
```

Then use left and right after i .

Skip duplicates to avoid repeated triplets.

19.5 C++ Code

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> result;

        sort(nums.begin(), nums.end());

        for (int i = 0; i < nums.size(); i++) {
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }

            int left = i + 1;
            int right = nums.size() - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (sum == 0) {
                    result.push_back({nums[i], nums[left], nums[right]});

                    left++;
                    right--;

                    while (left < right && nums[left] == nums[left - 1]) {
                        left++;
                    }

                    while (left < right && nums[right] == nums[right + 1]) {
                        right--;
                    }
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }

        return result;
    }
};
```

```
}  
};
```

19.6 Code Explanation

```
sort(nums.begin(), nums.end());
```

Enables two-pointer movement and duplicate skipping.

```
if (i > 0 && nums[i] == nums[i - 1]) {  
    continue;  
}
```

Skip duplicate fixed values.

```
left = i + 1;  
right = nums.size() - 1;
```

Search only after fixed index.

```
while (left < right && nums[left] == nums[left - 1])
```

Skip duplicate left values after storing valid triplet.

19.7 Dry Run

Input:

```
nums = {-1, 0, 1, 2, -1, -4}
```

Sorted:

```
{-4, -1, -1, 0, 1, 2}
```

For i = 1, nums[i] = -1:

2	5	{-1,-1,2}	0	store
3	4	{-1,0,1}	0	store

Result:

```
{{-1,-1,2}, {-1,0,1}}
```

19.8 Complexity

Time: $O(n^2)$

Sorting is $O(n \log n)$, but nested two-pointer scan dominates.

Space: $O(1)$ extra, ignoring output.

19.9 Edge Cases

- Fewer than 3 elements
- All zeroes
- No valid triplet

- Duplicates
- Large values, possible overflow

For overflow-safe version:

```
long long sum = (long long)nums[i] + nums[left] + nums[right];
```

19.10 Common Mistakes

- Not sorting
- Not skipping duplicates
- Starting left at 0
- Reusing same element
- Moving only one pointer after finding a triplet

19.11 Similar Problems

- 3Sum Closest
- 4Sum
- 3Sum Smaller
- 4Sum II, hashing variation

20. Full Problem Walkthrough 9 — LeetCode 977: Squares of a Sorted Array

20.1 Problem Summary

Given sorted integer array, return sorted squares.

Example:

```
nums = {-4, -1, 0, 3, 10}
```

Squares:

```
{16, 1, 0, 9, 100}
```

Output:

```
{0, 1, 9, 16, 100}
```

20.2 How to Identify This Pattern

Array is sorted, but negative numbers can produce large squares.

Largest square is at either end.

Use two pointers and fill output from the back.

20.3 Brute Force Idea

Square every number, then sort.

Time: $O(n \log n)$

20.4 Optimized Intuition

Compare absolute values at both ends.
Put larger square at the end of result.

20.5 C++ Code

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& nums) {
        int n = nums.size();

        vector<int> result(n);

        int left = 0;
        int right = n - 1;
        int pos = n - 1;

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if (leftSquare > rightSquare) {
                result[pos] = leftSquare;
                left++;
            } else {
                result[pos] = rightSquare;
                right--;
            }

            pos--;
        }

        return result;
    }
};
```

20.6 Dry Run

nums = {-4, -1, 0, 3, 10}

0	4	16	100	100	{_,_,_,100}
0	3	16	9	16	{_,_,16,100}

left	right	leftSquare	rightSquare	placed	result
1	3	1	9	9	{_,_,9,16,100}
1	2	1	0	1	{_,1,9,16,100}
2	2	0	0	0	{0,1,9,16,100}

20.7 Complexity

Time: $O(n)$

Space: $O(n)$

20.8 Edge Cases

- All positive
 - All negative
 - Contains zero
 - One element
-

20.9 Common Mistakes

- Filling from front
 - Using $\text{left} < \text{right}$ instead of $\text{left} \leq \text{right}$
 - Forgetting negative values can have large squares
-

21. Full Problem Walkthrough 10 — LeetCode 75: Sort Colors

21.1 Problem Summary

Given array with only 0, 1, and 2, sort it in-place.

Example:

```
nums = {2, 0, 2, 1, 1, 0}
```

Output:

```
{0, 0, 1, 1, 2, 2}
```

21.2 How to Identify This Pattern

Problem asks:

- sort only 0, 1, 2
- in-place
- one pass desired

Use Dutch National Flag algorithm.

21.3 Core Intuition

Maintain three regions:

```

[0 ... low-1]    = 0s
[low ... mid-1]   = 1s
[mid ... high]    = unknown
[high+1 ... n-1] = 2s

```

Process nums[mid].

- If 0: swap with low, move both low and mid
- If 1: move mid
- If 2: swap with high, move high only

21.4 C++ Code

```

class Solution {
public:
    void sortColors(vector<int>& nums) {
        int low = 0;
        int mid = 0;
        int high = nums.size() - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                swap(nums[low], nums[mid]);
                low++;
                mid++;
            } else if (nums[mid] == 1) {
                mid++;
            } else {
                swap(nums[mid], nums[high]);
                high--;
            }
        }
    }
};

```

21.5 Dry Run

nums = {2, 0, 2, 1, 1, 0}

0	0	5	2	swap mid/high, high--	{0,0,2,1,1,2}
0	0	4	0	swap low/mid, low++, mid++	{0,0,2,1,1,2}

low	mid	high	nums[mid]	action	array
1	1	4	0	swap low/mid, low++, mid++	{0,0,2,1,1,2}
2	2	4	2	swap mid/high, high--	{0,0,1,1,2,2}
2	2	3	1	mid++	{0,0,1,1,2,2}
2	3	3	1	mid++	{0,0,1,1,2,2}

21.6 Complexity

Time: $O(n)$

Space: $O(1)$

21.7 Edge Cases

- All 0s
 - All 1s
 - All 2s
 - Already sorted
 - Reverse sorted
 - Empty array
-

21.8 Common Mistake

After swapping with high, do not move mid.

Wrong:

```
swap(nums[mid], nums[high]);
high--;
mid++;
```

Why wrong?

The new value at mid has not been processed yet.

Correct:

```
swap(nums[mid], nums[high]);
high--;
```

21.9 Similar Problems

- Sort Array By Parity
 - Move Zeroes
 - Partition Array
 - QuickSort partition
-

22. Full Problem Walkthrough 11 — LeetCode 42: Trapping Rain Water

22.1 Problem Summary

Given elevation map height, compute how much rain water can be trapped.

Example:

```
height = {0,1,0,2,1,0,1,3,2,1,2,1}
```

Output:

```
6
```

22.2 How to Identify This Pattern

Water at each index depends on:

```
min(maxLeft, maxRight) - height[i]
```

Two pointers can optimize prefix/suffix max idea to $O(1)$ space.

22.3 Brute Force Idea

For each index:

16. Find max left
17. Find max right
18. Compute trapped water

Time: $O(n^2)$

22.4 Prefix/Suffix Idea

Precompute:

```
leftMax[i]  
rightMax[i]
```

Time: $O(n)$

Space: $O(n)$

22.5 Two-Pointer Intuition

Use:

```
left, right  
leftMax, rightMax
```

If $\text{height}[\text{left}] < \text{height}[\text{right}]$, then left side is limited by leftMax, because right boundary is already taller enough.

Process left.

Otherwise process right.

22.6 C++ Code

```

class Solution {
public:
    int trap(vector<int>& height) {
        int left = 0;
        int right = height.size() - 1;

        int leftMax = 0;
        int rightMax = 0;

        int water = 0;

        while (left < right) {
            if (height[left] < height[right]) {
                leftMax = max(leftMax, height[left]);
                water += leftMax - height[left];
                left++;
            } else {
                rightMax = max(rightMax, height[right]);
                water += rightMax - height[right];
                right--;
            }
        }

        return water;
    }
};

```

22.7 Dry Run

Short example:

height = {0, 1, 0, 2}

0	3	0	2	0	0	process left
1	3	1	2	1	0	process left
2	3	0	2	1	1	add 1
3	3	stop			1	

Output: 1

22.8 Correctness Explanation

At each step, process the side with smaller height. That side's water is determined by its own max boundary because the other side already provides a boundary at least as high as the current smaller side.

22.9 Complexity

Time: $O(n)$

Space: $O(1)$

22.10 Edge Cases

- Fewer than 3 bars
 - Strictly increasing heights
 - Strictly decreasing heights
 - All same height
 - Multiple valleys
-

22.11 Common Mistakes

- Thinking only adjacent bars matter
 - Not updating max before adding water
 - Processing the wrong side
 - Using $O(n^2)$ brute force in interview
-

22.12 Similar Problems

- Container With Most Water
 - Trapping Rain Water II, heap
 - Largest Rectangle in Histogram, monotonic stack
-

23. Full Problem Walkthrough 12 — LeetCode 680: Valid Palindrome II

23.1 Problem Summary

Return true if string can become a palindrome after deleting at most one character.

Example:

`s = "abca"`

Output:

`true`

Delete b or c.

23.2 How to Identify This Pattern

Problem says:

- palindrome
- allowed one deletion

Use two pointers.

At first mismatch, try skipping either left or right.

23.3 C++ Code

```
class Solution {
public:
    bool check(string& s, int left, int right) {
        while (left < right) {
            if (s[left] != s[right]) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }

    bool validPalindrome(string s) {
        int left = 0;
        int right = s.size() - 1;

        while (left < right) {
            if (s[left] != s[right]) {
                return check(s, left + 1, right) ||
                    check(s, left, right - 1);
            }

            left++;
            right--;
        }

        return true;
    }
};
```

23.4 Key Insight

At the first mismatch, only two deletions can help:

19. delete left character
20. delete right character

If neither works, impossible.

23.5 Dry Run

```
s = "abca"
```

Compare:

0	3	a/a	match
1	2	b/c	mismatch

Try:

```
check("abca", 2, 2) // delete b -> true  
check("abca", 1, 1) // delete c -> true
```

Return true.

23.6 Complexity

Time: $O(n)$

At most one extra palindrome scan.

Space: $O(1)$

23.7 Edge Cases

- Already palindrome
- One mismatch
- Multiple mismatches
- One character
- Two characters

23.8 Similar Problems

- Valid Palindrome
- Palindrome Partitioning
- Longest Palindromic Substring

24. Full Problem Walkthrough 13 — LeetCode 881: Boats to Save People

24.1 Problem Summary

Each boat can carry at most two people with total weight at most limit.

Return minimum number of boats.

Example:

```
people = {3, 2, 2, 1}  
limit = 3
```

Output:

3

24.2 How to Identify This Pattern

Problem asks:

- pair people
- minimize boats
- each boat holds at most two
- weights can be sorted

Use sorting + two pointers.

24.3 Greedy Intuition

Sort weights.

The heaviest remaining person must take a boat.

Try to pair them with the lightest remaining person.

If they fit, pair them.

Otherwise, heaviest goes alone.

24.4 C++ Code

```
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {
        sort(people.begin(), people.end());

        int left = 0;
        int right = people.size() - 1;

        int boats = 0;

        while (left <= right) {
            if (people[left] + people[right] <= limit) {
                left++;
            }

            right--;
            boats++;
        }

        return boats;
    }
};
```

24.5 Dry Run

```
people = {3, 2, 2, 1}
limit = 3
```

Sorted:

{1, 2, 2, 3}

0	3	$1 + 3 > 3$	3 alone	1
0	2	$1 + 2 \leq 3$	pair	2
1	1	2 alone	boat	3

Return 3.

24.6 Correctness Explanation

The heaviest person must be placed in some boat. Pairing them with the lightest possible person is always safe if they fit. If the lightest cannot fit with them, no one else can, so the heaviest must go alone.

24.7 Complexity

Time: $O(n \log n)$, due to sorting

Space: $O(1)$ extra, ignoring sorting internals

24.8 Edge Cases

- One person
- Everyone pairs perfectly
- No one can pair
- All same weights
- Heaviest exactly equals limit

24.9 Similar Problems

- Container With Most Water
- Assign Cookies
- Two Sum II
- Minimum Number of Boats style greedy

25. Full Problem Walkthrough 14 — LeetCode 844: Backspace String Compare

25.1 Problem Summary

Given two strings s and t , where $\#$ means backspace, return true if they are equal after processing.

Example:

$s = \text{"ab\#c"}$
 $t = \text{"ad\#c"}$

Both become:

"ac"

Output:

true

25.2 How to Identify This Pattern

This can be solved with stack.

But there is also a two-pointer solution from the end.

Since backspace affects previous characters, scanning backward lets us count skips.

25.3 Stack Solution

```
class Solution {
public:
    string build(string s) {
        string result;

        for (char c : s) {
            if (c == '#') {
                if (!result.empty()) {
                    result.pop_back();
                }
            } else {
                result.push_back(c);
            }
        }

        return result;
    }

    bool backspaceCompare(string s, string t) {
        return build(s) == build(t);
    }
};
```

Time: $O(n + m)$

Space: $O(n + m)$

25.4 Two-Pointer $O(1)$ Space Solution

```
class Solution {
public:
    int getNextValidIndex(string& s, int index) {
```

```

int skip = 0;

while (index >= 0) {
    if (s[index] == '#') {
        skip++;
        index--;
    } else if (skip > 0) {
        skip--;
        index--;
    } else {
        break;
    }
}

return index;
}

bool backspaceCompare(string s, string t) {
    int i = s.size() - 1;
    int j = t.size() - 1;

    while (i >= 0 || j >= 0) {
        i = getNextValidIndex(s, i);
        j = getNextValidIndex(t, j);

        if (i < 0 && j < 0) {
            return true;
        }

        if (i < 0 || j < 0) {
            return false;
        }

        if (s[i] != t[j]) {
            return false;
        }

        i--;
        j--;
    }

    return true;
}

```

```
}  
};
```

25.5 Key Idea

Scanning backward:

- # means skip one future valid character to the left
- skip counts how many characters to ignore

25.6 Complexity

Time: $O(n + m)$

Each character is processed at most once.

Space: $O(1)$

25.7 Edge Cases

- Extra backspaces
- Empty final strings
- One string empty after processing
- Consecutive #
- Strings already equal

25.8 Similar Problems

- Remove All Adjacent Duplicates
- Baseball Game
- Simplify Path
- Valid Parentheses

26. Mini Problem 1 — LeetCode 345: Reverse Vowels of a String

Pattern

Two pointers skipping non-vowels.

```
class Solution {  
public:  
    bool isVowel(char c) {  
        c = tolower(c);  
  
        return c == 'a' || c == 'e' || c == 'i' ||  
               c == 'o' || c == 'u';  
    }  
  
    string reverseVowels(string s) {  
        int left = 0;  
        int right = s.size() - 1;
```

```

while (left < right) {
    while (left < right && !isVowel(s[left])) {
        left++;
    }

    while (left < right && !isVowel(s[right])) {
        right--;
    }

    swap(s[left], s[right]);
    left++;
    right--;
}

return s;
}
};

```

Key idea:

Only vowels participate in swaps.

27. Mini Problem 2 — LeetCode 917: Reverse Only Letters

Pattern

Two pointers skipping non-letters.

```

class Solution {
public:
    string reverseOnlyLetters(string s) {
        int left = 0;
        int right = s.size() - 1;

        while (left < right) {
            while (left < right && !isalpha(s[left])) {
                left++;
            }

            while (left < right && !isalpha(s[right])) {
                right--;
            }

```

```
        swap(s[left], s[right]);
        left++;
        right--;
    }

    return s;
}
};
```

28. Mini Problem 3 — LeetCode 80: Remove Duplicates from Sorted Array II

Pattern

Read/write pointer with at most two copies.

```
class Solution {
public:
    int removeDuplicates(vector<int>& nums) {
        int write = 0;

        for (int x : nums) {
            if (write < 2 || x != nums[write - 2]) {
                nums[write] = x;
                write++;
            }
        }

        return write;
    }
};
```

Key idea:

Allow current value if it does not equal the value two positions behind in the kept portion.

29. Mini Problem 4 — LeetCode 443: String Compression

Pattern

Read/write pointer.

```

class Solution {
public:
    int compress(vector<char>& chars) {
        int write = 0;
        int read = 0;

        while (read < chars.size()) {
            char current = chars[read];
            int count = 0;

            while (read < chars.size() && chars[read] == current) {
                read++;
                count++;
            }

            chars[write] = current;
            write++;

            if (count > 1) {
                string countStr = to_string(count);

                for (char c : countStr) {
                    chars[write] = c;
                    write++;
                }
            }
        }

        return write;
    }
};

```

30. Mini Problem 5 — LeetCode 986: Interval List Intersections

Pattern

Two pointers over two sorted interval lists.

```

class Solution {
public:
    vector<vector<int>> intervalIntersection(vector<vector<int>>& firstList,

```

```

        vector<vector<int>>& secondList) {
    vector<vector<int>> result;

    int i = 0;
    int j = 0;

    while (i < firstList.size() && j < secondList.size()) {
        int start = max(firstList[i][0], secondList[j][0]);
        int end = min(firstList[i][1], secondList[j][1]);

        if (start <= end) {
            result.push_back({start, end});
        }

        if (firstList[i][1] < secondList[j][1]) {
            i++;
        } else {
            j++;
        }
    }

    return result;
}
};

```

Key idea:

The interval that ends earlier cannot overlap future intervals anymore.

31. Common Edge Cases for Two Pointers

31.1 Empty Array or String

Be careful with:

```
int right = nums.size() - 1;
```

If empty, this can be problematic due to unsigned behavior.

Safer:

```
int right = (int)nums.size() - 1;
```

or check empty first.

31.2 Single Element

Usually:

```
while (left < right)
```

does not run, which is fine.

31.3 Pointer Crossing

Know when to use:

```
left < right
```

versus:

```
left <= right
```

Use `left < right` when selecting two distinct elements.

Use `left <= right` when every element must be processed, like sorted squares.

31.4 Duplicates

Important for:

- 3Sum
- 4Sum
- Remove Duplicates
- Sort Colors

Duplicate skipping must happen carefully.

31.5 Equal Values

For Two Sum sorted, equal values may be valid if they are at different indices.

Example:

```
numbers = {3, 3}
```

```
target = 6
```

31.6 Negative Numbers

Important for:

- 3Sum
- Sorted Squares
- Pair sums

Two pointers still works if sorted.

31.7 Original Index Requirement

Sorting destroys original positions.

For problems requiring original indices, use hash map or store pairs.

31.8 Overflow in Sum

For large values:

```
long long sum = (long long)nums[left] + nums[right];
```

For 3Sum:

```
long long sum = (long long)nums[i] + nums[left] + nums[right];
```

32. Common Mistakes and Wrong Code

Mistake 1 — Using Two Pointers on Unsorted Pair Sum

Wrong:

```
int left = 0;
int right = nums.size() - 1;

while (left < right) {
    int sum = nums[left] + nums[right];

    if (sum < target) {
        left++;
    } else {
        right--;
    }
}
```

This only works if nums is sorted.

Mistake 2 — Moving Wrong Pointer

For sorted sum:

```
if (sum < target) {
    left++;
} else {
    right--;
}
```

Do not reverse this.

Mistake 3 — Infinite Loop

Every branch must either:

- return
 - increment left
 - decrement right
-

Mistake 4 — Not Skipping Duplicates in 3Sum

Can produce duplicate triplets.

Need skip fixed duplicates and pointer duplicates.

Mistake 5 — Sorting When Position Matters

Wrong for:

- Container With Most Water
 - Stock prices
 - subarray problems
-

Mistake 6 — Using $left < right$ When Last Element Must Be Processed

Sorted Squares needs:

```
while (left <= right)
```

Mistake 7 — Moving mid After Swapping With high in Sort Colors

Wrong:

```
swap(nums[mid], nums[high]);  
high--;  
mid++;
```

Correct:

```
swap(nums[mid], nums[high]);  
high--;
```

33. Interview Explanation Templates

33.1 Sorted Two Sum

Since the array is sorted, I use two pointers at both ends. If the sum is too small, I move the left pointer to increase it. If the sum is too large, I move the right pointer to decrease it. This gives $O(n)$ time and $O(1)$ space.

33.2 Palindrome

A palindrome must match symmetrically. I compare characters from both ends and move inward. If a mismatch occurs, the string is not a palindrome unless the problem allows a deletion.

33.3 Read/Write Pointer

I use a read pointer to scan every element and a write pointer to store the next valid element. This lets me modify the array in-place while preserving order.

33.4 Container With Most Water

I start with the widest container. The shorter line limits the area. Since moving either pointer reduces width, the only chance to improve area is to move the shorter line and hope for a taller one.

33.5 3Sum

I sort the array, then fix one number and use two pointers to find two remaining numbers that complete the target sum. Sorting allows me to move pointers based on sum and skip duplicates.

34. Complexity Patterns

Palindrome check	$O(n)$	$O(1)$
Reverse string	$O(n)$	$O(1)$
Two Sum sorted	$O(n)$	$O(1)$
Remove element	$O(n)$	$O(1)$
Remove duplicates	$O(n)$	$O(1)$
Move zeroes	$O(n)$	$O(1)$
Container With Most Water	$O(n)$	$O(1)$
3Sum	$O(n^2)$	$O(1)$ extra
4Sum	$O(n^3)$	$O(1)$ extra
Sorted Squares	$O(n)$	$O(n)$
Sort Colors	$O(n)$	$O(1)$
Trapping Rain Water	$O(n)$	$O(1)$

35. Time-Boxed Practice Guidance

Easy Two-Pointer Problems

Target:

5–12 minutes

Examples:

- Reverse String
 - Valid Palindrome
 - Remove Element
 - Move Zeroes
-

Medium Core Problems

Target:

15–30 minutes

Examples:

- Two Sum II
 - Container With Most Water
 - 3Sum
 - Sort Colors
 - Sorted Squares
-

Harder Problems

Target:

30–45 minutes

Examples:

- Trapping Rain Water
 - 4Sum
 - Backspace String Compare $O(1)$ space
 - Interval List Intersections
-

36. Expanded Practice Set

36.1 Must-Do Easy

- 1.1. Valid Palindrome
 - 2.2. Reverse String
 - 3.3. Reverse Vowels of a String
 - 4.4. Reverse Only Letters
 - 5.5. Two Sum II — Input Array Is Sorted
 - 6.6. Remove Duplicates from Sorted Array
 - 7.7. Remove Element
 - 8.8. Move Zeroes
 - 9.9. Squares of a Sorted Array
 - 10.10. Valid Palindrome II
-

36.2 Must-Do Medium

- 11.11. Container With Most Water
 - 12.12. 3Sum
 - 13.13. 3Sum Closest
 - 14.14. 4Sum
 - 15.15. Sort Colors
 - 16.16. Remove Duplicates from Sorted Array II
 - 17.17. String Compression
 - 18.18. Boats to Save People
 - 19.19. Interval List Intersections
 - 20.20. Backspace String Compare
-

36.3 Strong Interview Practice

- 21.21. Trapping Rain Water
- 22.22. Next Permutation
- 23.23. 3Sum Smaller
- 24.24. Subarray Product Less Than K, sliding window overlap

25.25.	Longest Word in Dictionary through Deleting
26.26.	Find K Closest Elements
27.27.	Rotate Array
28.28.	Reverse Words in a String II
29.29.	Sort Transformed Array
30.30.	Longest Mountain in Array

36.4 Hard / Stretch

31.31.	Trapping Rain Water
32.32.	4Sum
33.33.	Substring with Concatenation of All Words, hashing + sliding window
34.34.	Minimum Window Substring, sliding window
35.35.	Subarrays with K Different Integers, sliding window
36.36.	Interval List Intersections
37.37.	Smallest Range Covering Elements from K Lists, heap variation

37. Suggested Study Order

Step 1 — Basic Opposite-End Pointers

- 21. Reverse String
- 22. Valid Palindrome
- 23. Reverse Vowels of a String
- 24. Reverse Only Letters

Goal:

- Learn inward pointer movement.
-

Step 2 — Sorted Pair Logic

- 25. Two Sum II
- 26. Squares of a Sorted Array
- 27. Boats to Save People

Goal:

- Learn how sorted order controls pointer movement.
-

Step 3 — Read/Write Pointer

- 28. Remove Element
- 29. Move Zeroes
- 30. Remove Duplicates from Sorted Array
- 31. Remove Duplicates from Sorted Array II
- 32. String Compression

Goal:

- Master in-place compaction.
-

Step 4 — Boundary Optimization

- 33. Container With Most Water
- 34. Trapping Rain Water

Goal:

- Learn why moving one boundary is safe.

Step 5 — Fixed Index + Two Pointers

- 35. 3Sum
- 36. 3Sum Closest
- 37. 4Sum

Goal:

- Learn sort + fixed element + duplicate skipping.
-

Step 6 — Partitioning

- 38. Sort Colors
- 39. Sort Array By Parity
- 40. Partition Array

Goal:

- Learn multi-pointer partitioning.
-

38. Pattern Decision Table

Sorted array + target pair	left = 0, right = n - 1	Two Sum II
Palindrome	compare ends	Valid Palindrome
Reverse in-place	swap ends	Reverse String
Remove elements	read/write	Remove Element
Remove duplicates sorted	read/write	Remove Duplicates
Move zeroes	read/write	Move Zeroes
Max area from two boundaries	move shorter side	Container With Most Water
Triplets	sort + fixed i + left/right	3Sum
Sorted squares	fill from back	Squares of Sorted Array
Three-way partition	low/mid/high	Sort Colors
Water trapping	left/right + maxes	Trapping Rain Water
Backspaces from end	reverse scan with skip	Backspace String Compare

39. Final Cheat Sheet

Opposite Pointers

```
int left = 0;
int right = nums.size() - 1;

while (left < right) {
    // process
    left++;
    right--;
}
```

Sorted Two Sum

```
while (left < right) {
    int sum = nums[left] + nums[right];

    if (sum == target) {
        return {left, right};
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
```

Palindrome

```
while (left < right) {
    if (s[left] != s[right]) {
        return false;
    }

    left++;
    right--;
}

return true;
```

Read/Write Pointer

```
int write = 0;

for (int read = 0; read < nums.size(); read++) {
    if (/* keep */) {
        nums[write] = nums[read];
    }
}
```

```
        write++;
    }
}

return write;
```

Remove Duplicates Sorted

```
if (nums.empty()) {
    return 0;
}

int write = 1;

for (int read = 1; read < nums.size(); read++) {
    if (nums[read] != nums[write - 1]) {
        nums[write] = nums[read];
        write++;
    }
}

return write;
```

3Sum Skeleton

```
sort(nums.begin(), nums.end());

for (int i = 0; i < nums.size(); i++) {
    if (i > 0 && nums[i] == nums[i - 1]) {
        continue;
    }

    int left = i + 1;
    int right = nums.size() - 1;

    while (left < right) {
        long long sum = (long long)nums[i] + nums[left] + nums[right];

        if (sum == 0) {
            result.push_back({nums[i], nums[left], nums[right]});

            left++;
            right--;
        }
    }
}
```

```

        while (left < right && nums[left] == nums[left - 1]) {
            left++;
        }

        while (left < right && nums[right] == nums[right + 1]) {
            right--;
        }
    } else if (sum < 0) {
        left++;
    } else {
        right--;
    }
}
}

```

Sort Colors

```

int low = 0;
int mid = 0;
int high = nums.size() - 1;

while (mid <= high) {
    if (nums[mid] == 0) {
        swap(nums[low], nums[mid]);
        low++;
        mid++;
    } else if (nums[mid] == 1) {
        mid++;
    } else {
        swap(nums[mid], nums[high]);
        high--;
    }
}

```

Trapping Rain Water

```

int left = 0;
int right = height.size() - 1;

int leftMax = 0;
int rightMax = 0;
int water = 0;

```

```

while (left < right) {
    if (height[left] < height[right]) {
        leftMax = max(leftMax, height[left]);
        water += leftMax - height[left];
        left++;
    } else {
        rightMax = max(rightMax, height[right]);
        water += rightMax - height[right];
        right--;
    }
}
}

```

40. What to Master Before Chapter 4

Before moving to Sliding Window, make sure you can:

- Explain why sorted arrays allow pointer movement
- Solve Two Sum II
- Reverse an array/string in-place
- Check palindromes using two pointers
- Use read/write pointer for in-place removal
- Remove duplicates from sorted array
- Move zeroes while preserving order
- Solve Container With Most Water
- Sort and solve 3Sum
- Skip duplicates correctly
- Solve Sorted Squares
- Understand Dutch National Flag for Sort Colors
- Know when $\text{left} < \text{right}$ vs $\text{left} \leq \text{right}$ is needed
- Avoid using two pointers incorrectly on unsorted arrays

41. Core Takeaway

Two Pointers is about using two positions to eliminate unnecessary work.

The core question is:

Can moving one pointer safely discard many impossible choices?

If yes, two pointers may work.

The biggest ideas are:

41. Sorted arrays give direction.
42. Palindromes use symmetry.
43. In-place filtering uses read/write.
44. Boundary problems often move the limiting side.
45. 3Sum and 4Sum use sorting + fixed index + two pointers.
46. Pointer movement must always be justified.

Most two-pointer solutions look simple after you understand **why each pointer moves**.

That “why” is what interviewers care about.
